

Intro to Deep Learning

Lecture 04: Generalization, Regularization, and LR Schedules

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. How do we recognize overfitting in curves?
2. Why is learning rate often the first thing to check?
3. What do LR schedules do?
4. How do weight decay, dropout, early stopping, and augmentation help?
5. What changes between `train()` and `eval()` ?
6. Why do we log experiments?

Lecture rhythm: diagnose curve -> choose intervention -> compare runs

Recap From Lecture 03

For classification, we choose:

```
output shape -> target format -> loss -> metric
```

Today:

```
training performance != real performance
```

We will refresh overfitting, then focus on practical controls.

Part 1

Generalization

Training well is not the same as generalizing well

Generalization Refresher

You already know the basic split:

```
train      -> update parameters  
validation -> tune/debug choices  
test       -> final evaluation
```

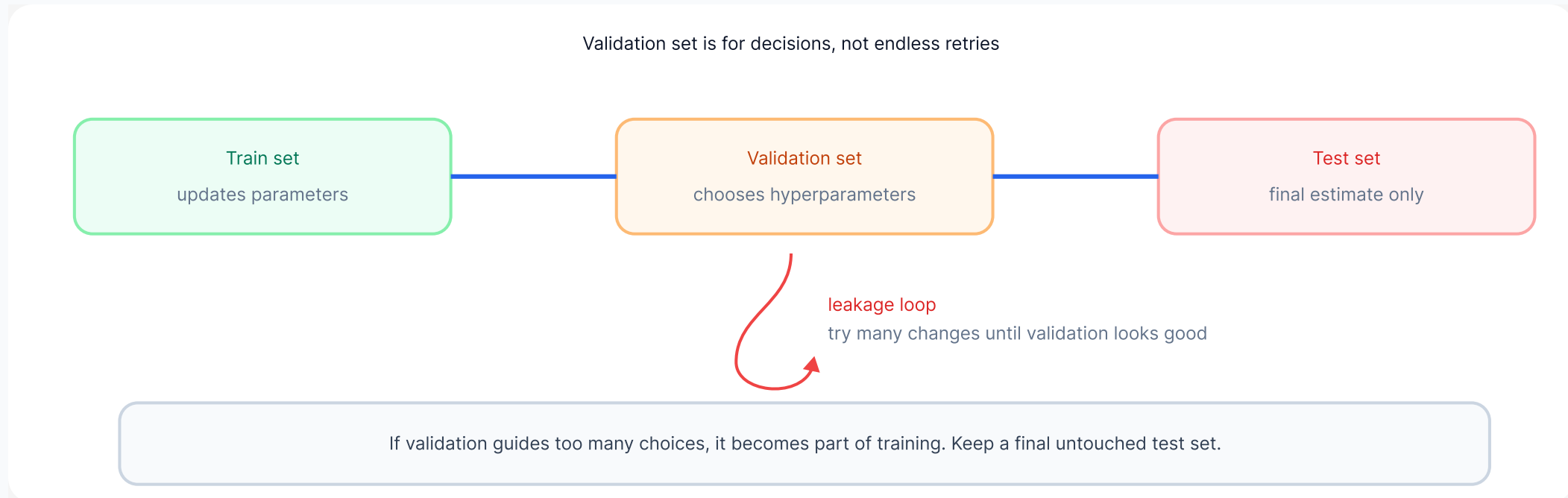
Generalization means:

```
good performance on data not used for parameter updates
```

Main warning:

```
do not repeatedly tune on the test set
```

Validation Leakage



The validation set can leak into modeling decisions.

If you keep trying ideas until validation improves, you are indirectly fitting validation.

Use validation for iteration; keep test for the final estimate.

Curves: Quick Diagnosis

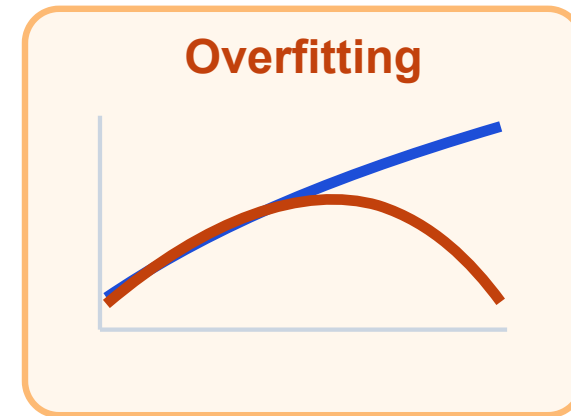
Read train and validation curves together



train bad, validation bad



both improve, gap is reasonable
— train — validation

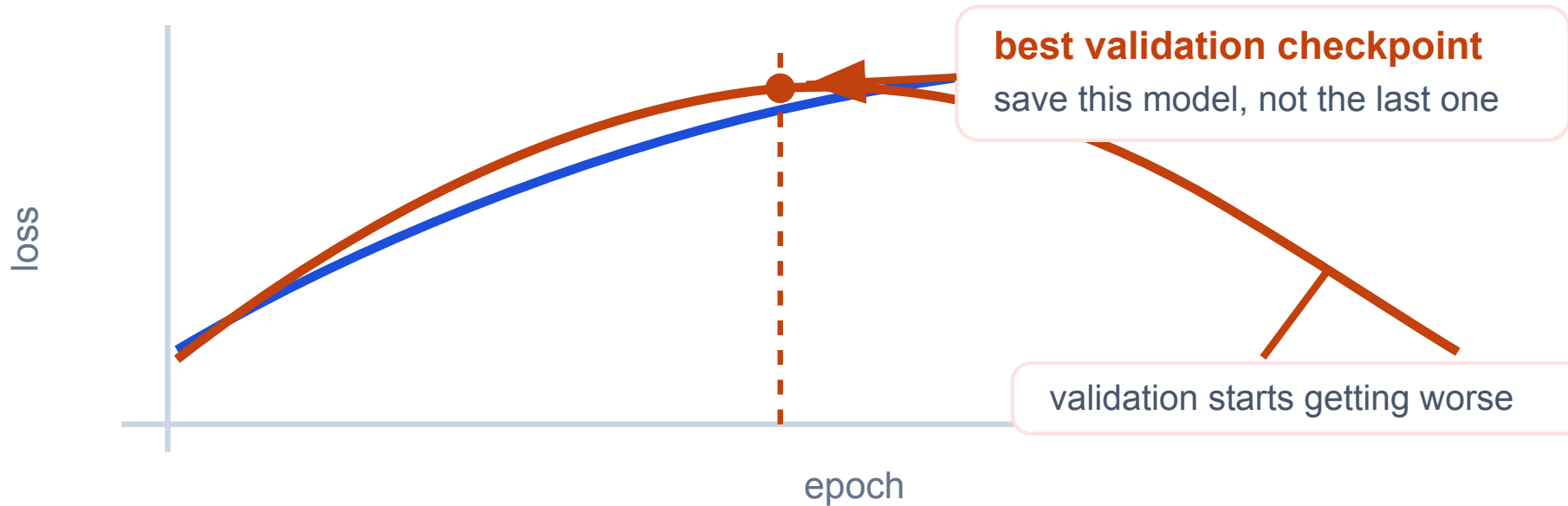


train improves, validation worsens

We only need this refresher to interpret today's experiments.

The Validation Loss Turn

The best model is often before the final epoch



This moment tells us when to stop and which checkpoint generalized best.

Part 2

Learning Rate

Optimization controls before architecture changes

Learning Rate Matters

Learning rate controls optimizer step size.

Too small:

slow learning; loss changes very little

Too large:

unstable learning; loss explodes or oscillates

Good LR:

loss decreases steadily

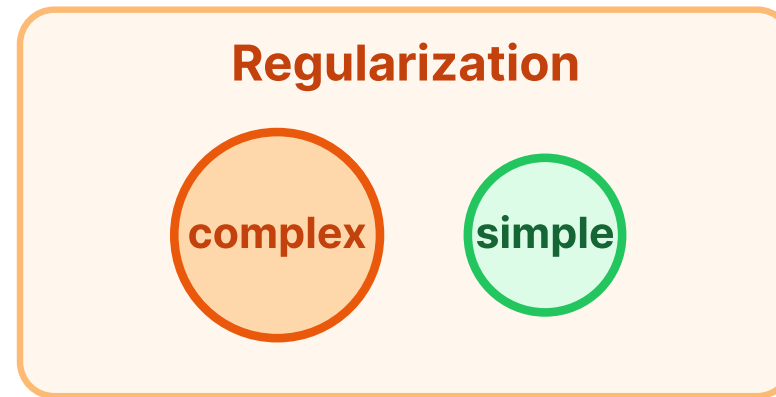
Before changing architecture, check the learning rate.

LR Is Not Regularization

Learning rate is optimization; regularization is preference



How big are optimizer steps?



Which solutions do we prefer?

They interact in practice:

too high LR can look like bad model
too low LR can look like underfitting

Learning Rate Schedules

An LR schedule changes learning rate during training.

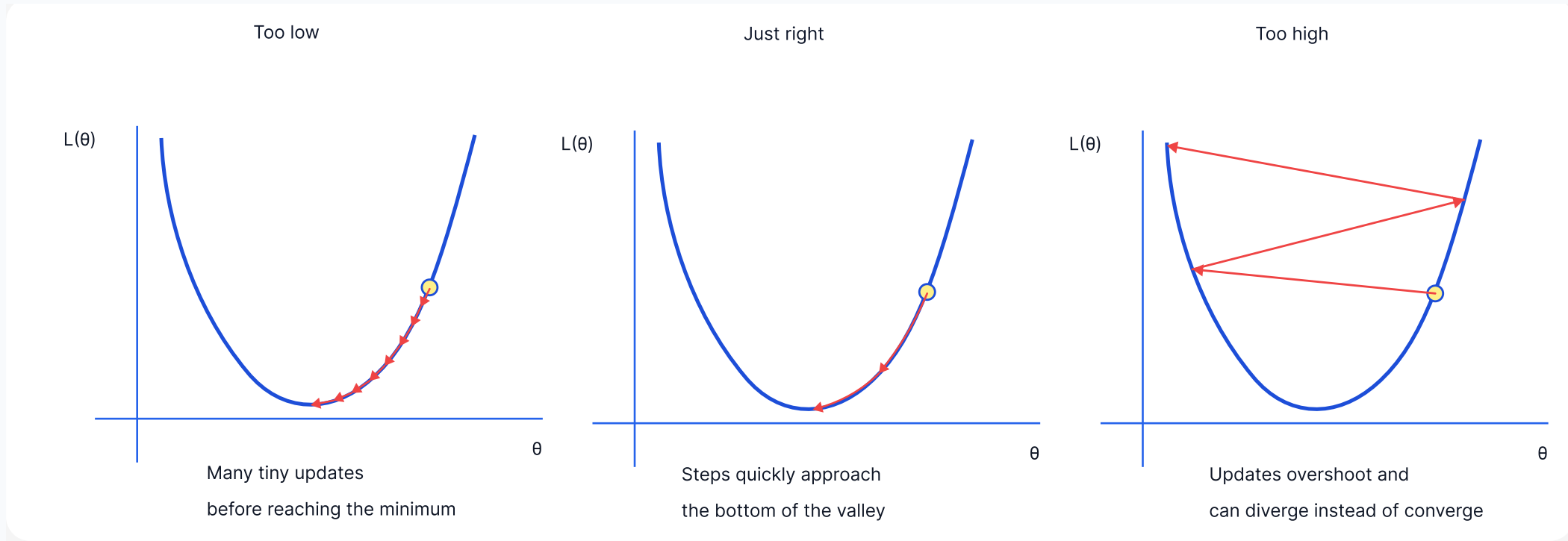
Common idea:

```
start with larger LR  
reduce it later
```

Why?

- make progress early
- refine solution later
- avoid bouncing around near good parameters

LR As Step Size



The gradient gives a direction.

The learning rate decides **how far** we move in that direction.

Why Change LR Over Time?

Training has different needs at different stages.

Early

Move quickly away from random initialization.

Need: exploration and fast progress.

Middle

Follow useful directions while avoiding instability.

Need: steady optimization.

Late

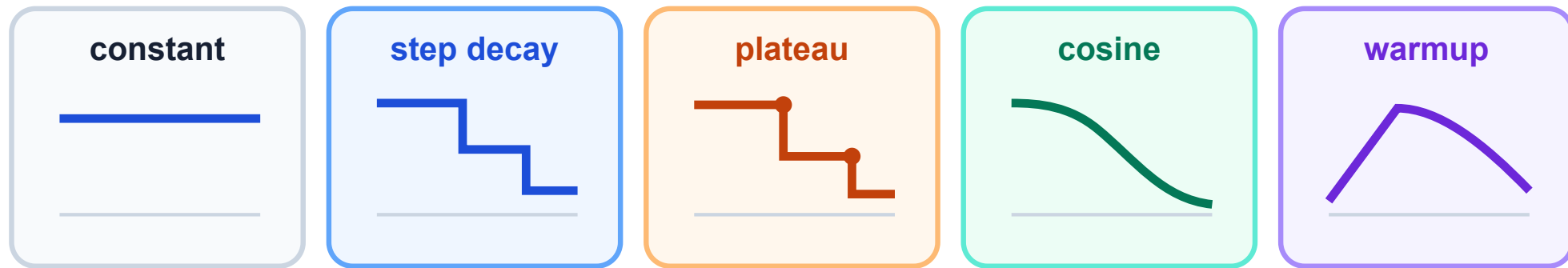
Refine the solution without bouncing around.

Need: smaller, careful steps.

Schedules encode this change of behavior.

Schedule Shapes

Learning-rate schedules change step size over time



Schedules help after the basic training setup already works.

Recognize the shape first. The exact API matters less than the idea.

Common LR Strategies

Strategy	Idea	Useful when...
Constant LR	Use one step size for the whole run.	You need a simple baseline or very short training.
Step decay	Train in phases, then suddenly reduce LR.	You know roughly when progress should become more careful.
Exponential decay	Continuously become more conservative.	You want a smooth, predictable decrease.
Cosine decay	Smoothly move from high LR to low LR.	You know the total training budget.

These are the usual first choices for standard supervised training.

Special LR Strategies

Strategy	Idea	Useful when...
Warmup	Start small, increase, then decay.	Early updates are unstable; common in large models.
Cyclical LR	Periodically increase LR again.	You want repeated exploration instead of pure convergence.
Plateau-based	Reduce LR when validation stops improving.	You do not know the right drop time in advance.

Use these when the beginning of training is unstable or when progress stalls unpredictably.

Which LR Strategy Should I Try?

Situation	First strategy to try
New baseline, short run	Constant LR
Training improves, then gets noisy	Decay LR later
Validation plateaus	Reduce on plateau
Fixed epoch budget	Cosine decay
Very unstable first epochs	Warmup
Large pretrained / Transformer-like model	Warmup + decay

Rule of thumb: first find a reasonable LR, then decide how it should change.

Schedules are useful **after** the basic setup works.

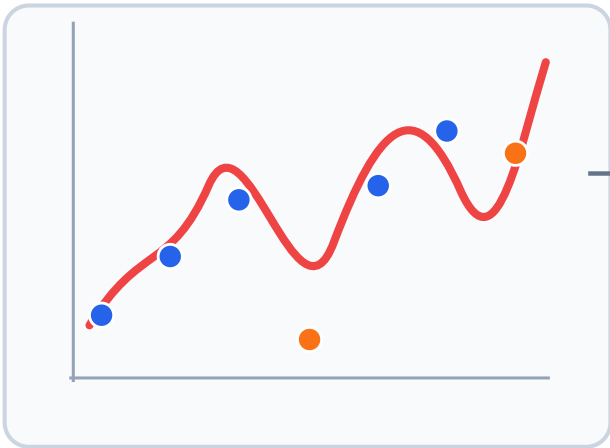
Part 3

Regularization

Prefer solutions that generalize better

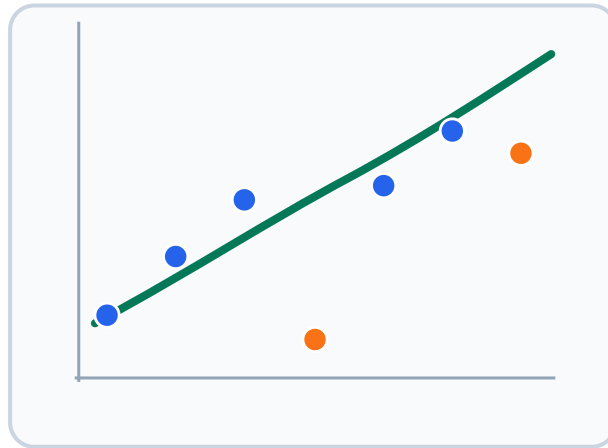
Why Do We Need Regularization?

Without regularization



fits noise too well

With regularization



captures stable pattern

The trade-off

A powerful network can memorize training quirks and label noise.

Regularization adds a preference: choose solutions that are simpler, smoother, or more robust.

Deep networks often have enough capacity to fit **training noise**.

Regularization adds a useful bias:

prefer solutions that still work on new data

Regularization

Regularization means adding constraints or noise so the model learns simpler, more robust patterns.

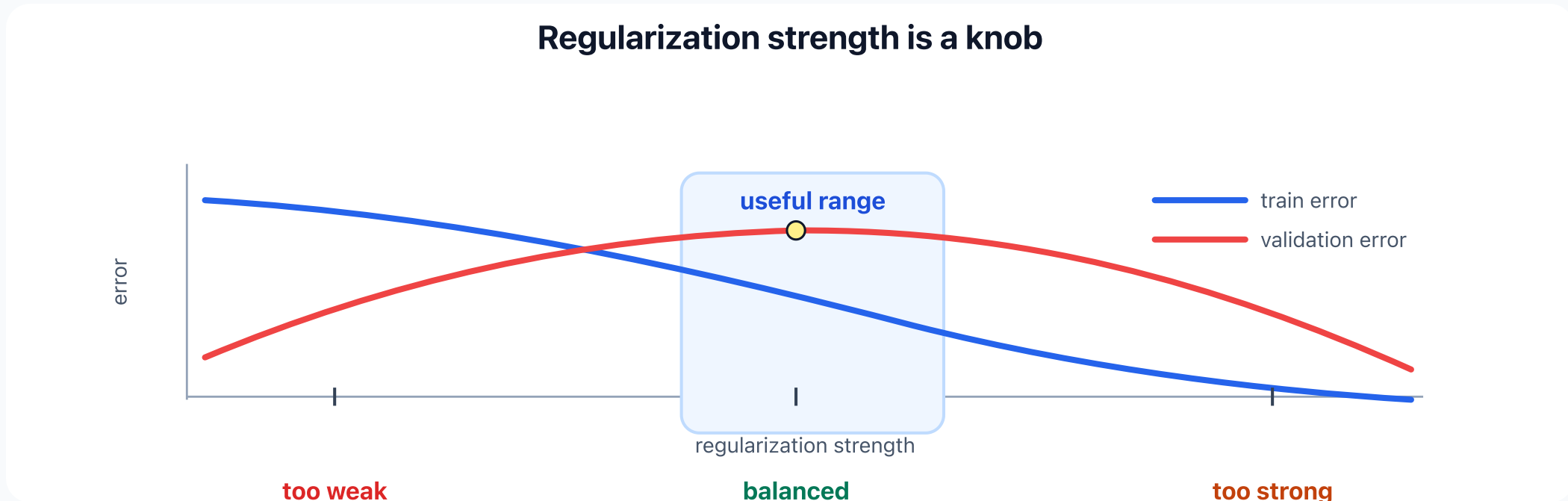
Goal:

```
reduce validation error  
not necessarily reduce training error
```

Regularization may make training accuracy lower.

That can be acceptable if validation performance improves.

Regularization Strength Is A Knob



Too little regularization can overfit.

Too much regularization can underfit.

Choose strength by validation performance, not by training accuracy alone.

Weight Decay

Weight decay penalizes large weights.

Objective:

$$L_{total} = L_{data} + \lambda ||W||^2$$

Intuition:

```
prefer smaller weights  
avoid overly sharp or specialized solutions
```

In PyTorch optimizers:

```
weight_decay=...
```

Dropout

Dropout randomly disables hidden units during training.

Training:

some activations are set to zero

Evaluation:

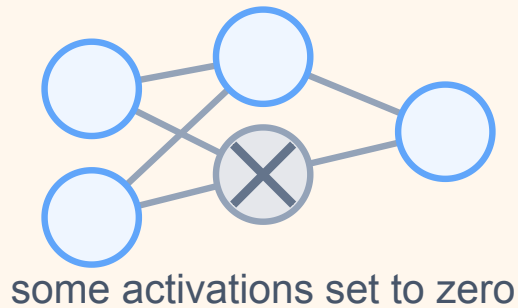
all units are used

This forces the network to avoid relying too much on one specific hidden unit.

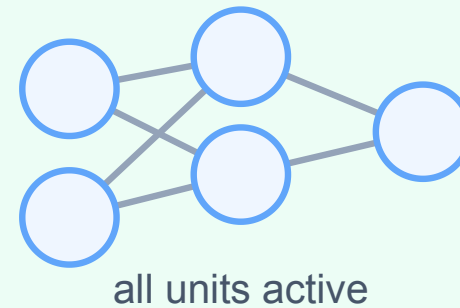
Dropout And `train/eval`

Dropout changes between `train()` and `eval()`

`model.train()`



`model.eval()`



Forgetting `model.eval()` changes validation behavior.

Dropout Intuition

Without dropout:

one feature can dominate

With dropout:

the model must build redundant useful representations

Common values:

$p = 0.1$ to 0.5

Too much dropout can cause underfitting.

Early Stopping

Early stopping saves the best validation model.

Basic idea:

```
if validation loss has not improved for N epochs:  
    stop training
```

It prevents training long after generalization has stopped improving.

Important:

```
restore best checkpoint, not final checkpoint
```

Part 4

Data Augmentation

Teach invariances we expect at test time

Data Augmentation

Data augmentation creates modified training examples.

For images:

- small rotations
- translations
- crops
- flips
- color changes

Goal:

teach invariances we expect at test time

Safe Augmentation

Augmentation must preserve the label



A transformed 6 should not become a 9 .

Train Only

Apply augmentation to training data only.

Training:

random transformations improve robustness

Validation/test:

measure performance on real unchanged data

If validation is randomly augmented, the metric becomes noisy and hard to compare.

Part 5

Reproducibility And Tracking

Memory is not an experiment log

Reproducibility

Deep learning experiments have many moving parts:

- random initialization
- data shuffling
- train/validation split
- augmentation randomness
- GPU nondeterminism

Use seeds when comparing runs.

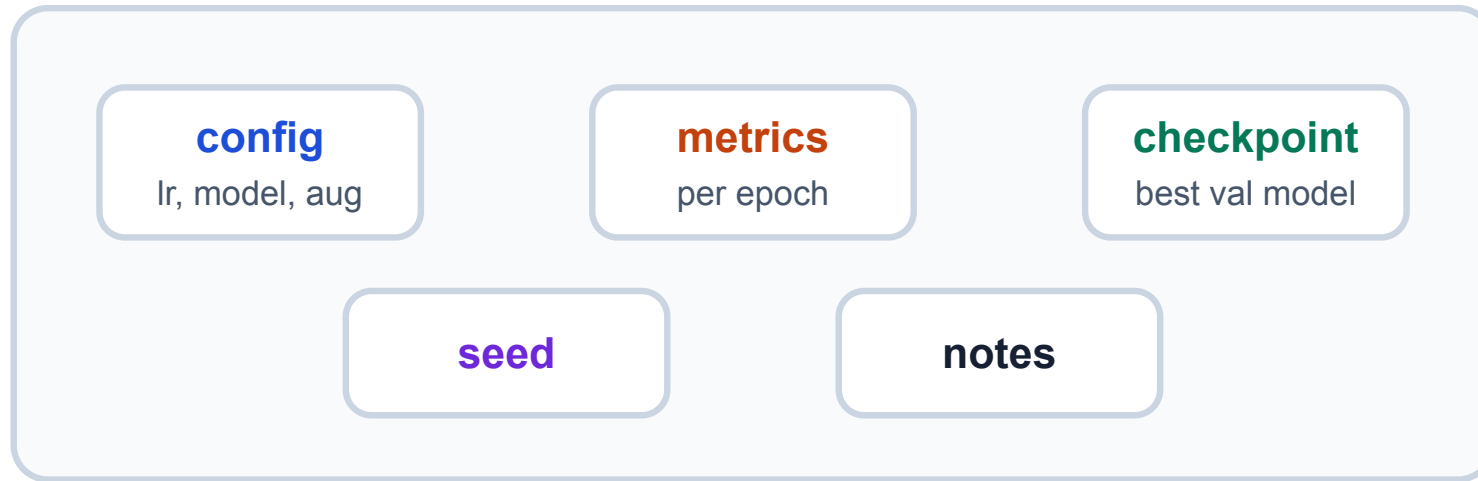
But remember:

one seed is not a scientific conclusion

Experiment Tracking

If you cannot remember what changed, you cannot learn from experiments.

A run record turns experiments into reusable knowledge



Even a simple CSV log is much better than memory.

Minimal Run Folder

Useful run structure:

```
runs/my_experiment/  
  config.json  
  metrics.csv  
  summary.json
```

`config.json` : what did I run?

`metrics.csv` : what happened each epoch?

`summary.json` : what should I remember?

Comparing Experiments

Change one main thing at a time.

Example sequence:

```
baseline  
baseline + dropout  
baseline + augmentation  
baseline + dropout + augmentation
```

Then compare:

- best validation accuracy
- train/validation gap
- stability
- training time

Part 6

Debugging Training

The curve chooses the next experiment

Debugging Decision Tree

Use the curve pattern to choose the next experiment



Do not start with dropout if both train and validation are bad.

Practical Checklist

When training a model, always look at:

1. train loss
2. validation loss
3. train metric
4. validation metric
5. learning rate
6. config and seed

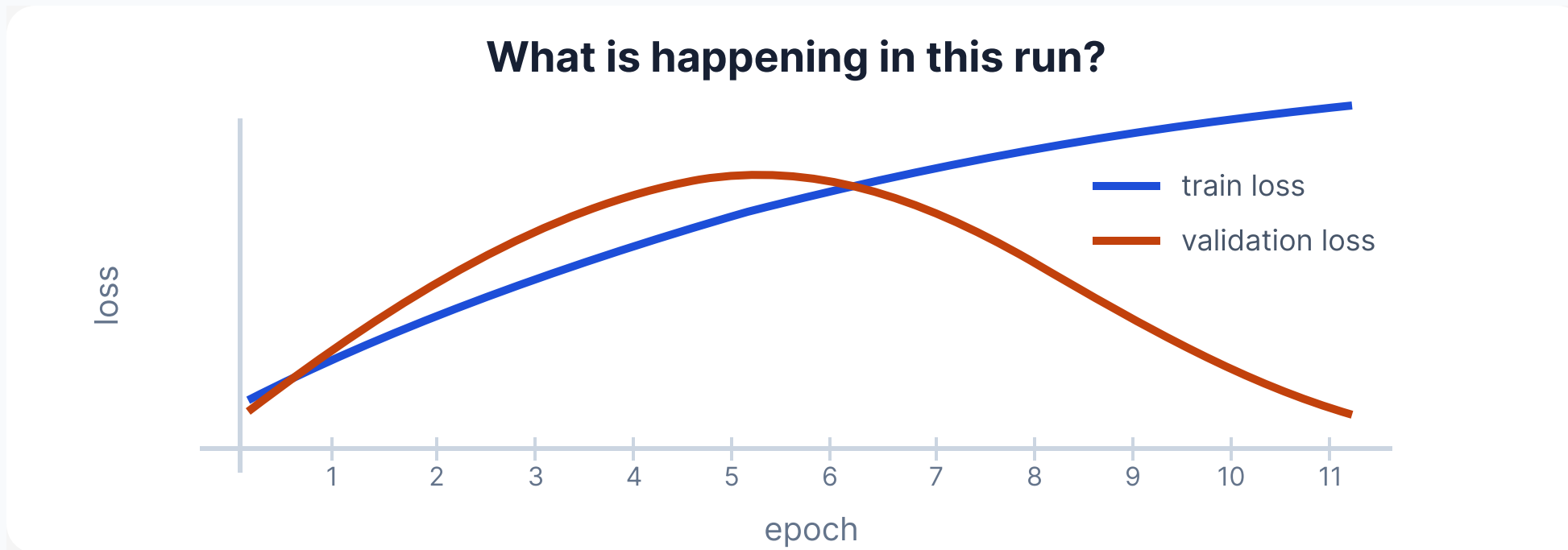
Without these, debugging becomes guesswork.

Part 7

Seminar Bridge

See overfitting, then control it

Mini Check



What is happening?

What epoch might contain the best model?

Answer

This is overfitting.

The model keeps improving on the training set but generalization gets worse.

The best checkpoint is probably around:

epoch 6

not epoch 20.

Mini Check

You add dropout.

Training accuracy becomes lower.

Validation accuracy becomes higher.

Is this good or bad?

Answer

Usually good.

Dropout can make the training task harder.

But if validation performance improves, the model generalizes better.

Remember:

we care about unseen data

What Will Happen In Seminar?

You will:

1. intentionally overfit a large MLP on small MNIST data
2. plot train/validation curves
3. add dropout
4. add data augmentation
5. compare runs
6. save `config.json`, `metrics.csv`, and `summary.json`

The goal is to see overfitting, not just hear about it.

Key Takeaways

1. Generalization is measured on data not used for updates.
2. Learning curves reveal underfitting and overfitting.
3. LR affects optimization; schedules change LR over time.
4. Regularization can reduce validation error even if train error increases.
5. Augmentation must preserve labels.
6. Logged experiments are reusable knowledge.

Next lecture:

CNNs from first principles